

PentESTGORage

keygen_me

🕒 Created	@Jan 16, 2021 6:07 PM
🏷 tags	reversing z3

We are given a binary and an endpoint to connect to as `35.184.207.253:1337`

```
> file keygen_me
keygen_me: ELF 64-bit LSB pie executable, x86-64, version 1 (SYSV), dynamically linked, interpreter /lib64/ld-linux-x86-64.so.2, BuildID[sha1]=f34cb69cc0c31583a75e159122a1c6277a564bee, for GNU/Linux 3.2.0, not stripped
```

Running the binary we are asked for a key, providing an invalid key prints `Invalid`

`key`

```
> ./keygen_me
Enter key: AAAA
Invalid key
```

Opening the binary in ghidra

The image displays a side-by-side comparison of assembly code and its decompiled C++ equivalent. The left pane, titled 'Listing: keygen_me', shows assembly instructions with their hex values, mnemonics, and comments. The right pane, titled 'C++ Decompile: main - (keygen_me)', shows the decompiled C++ code, which includes a main function that reads a key, checks if it's 'invalid', and prints a message if it's not.

```

Listing: keygen_me
001014d4 e8 b7 fb CALL open
001014d9 b9 00 01 MOV ECX,0x100
001014de ba 00 00 MOV EIX,0x0
001014e3 89 c6 MOV ESI,EAX
001014e5 bf 01 00 MOV EDI,0x1
001014ea e8 81 fb CALL sendfile
001014ef eb 0c JMP LAB_001014fd
001014f1 4b 8d 3d LEA EDI,[s_invalid_key_00102]
001014f8 e8 31 fb CALL puts
001014fd b8 00 00 MOV EAX,0x0
00101502 c9 LEAVE
00101503 c3 RET
00101504 66 ?? 66h f
00101505 2e ?? 2Eh .
00101506 0f ?? 0Fh .
00101507 1f ?? 1Fh .
00101508 84 ?? 84h .
00101509 00 ?? 00h .
0010150a 00 ?? 00h .
0010150b 00 ?? 00h .

C++ Decompile: main - (keygen_me)
1 undefined8 main(void)
2
3
4 {
5     int iVar1;
6     undefined8 uStack208;
7     undefined local_118 [264];
8     ssize_t local_10;
9
10    uStack208 = 0x101467;
11    printf("Enter key: ");
12    uStack208 = 0x101480;
13    local_10 = read(0,local_118,0xff);
14    local_118[local_10 + -1] = 0;
15    uStack208 = 0x1014ae;
16    iVar1 = check(local_118,local_10 + -1,local_10 + -1);
17    if (iVar1 == 0) {
18        uStack208 = 0x1014fd;
19        puts("Invalid key");
20    }
21    else {
22        uStack208 = 0x1014c3;
23        printf("Congrats here is your flag: ");
24        uStack208 = 0x1014d5;
25        iVar1 = open("/flag",0);
26        uStack208 = 0x1014ef;
27        sendfile(1,iVar1,(off_t *)0x0,0x100);
28    }
29    return 0;
30 }
31

```

we see that our input is being passed to the `check` function which checks the password. Opening the `check` function we find a lot of conditions which needs to be satisfied

The screenshot displays the Immunity Debugger interface with two main panes. The left pane shows assembly code with addresses, hex values, and mnemonics. The right pane shows the decompiled C++ code for the same function.

Assembly View (Left Pane):

```

00101414 40 0b  mov     eax, 0b
00101410 40 01  mov     eax, 01
0010141c 0f b4  mov     ecx, [edi+0f0b4]
0010141f 48 8b  mov     eax, [eax]
00101423 48 8b  mov     eax, [eax]
00101427 0f b4  mov     ecx, [edi+0f0b4]
0010142a 31 c2  scasd   ecx
0010142e 48 8b  mov     eax, [eax]
00101430 48 8b  mov     eax, [eax]
00101434 0f b4  mov     ecx, [edi+0f0b4]
00101437 31 dc  scasd   ecx
00101439 3c 42  cmp     al, 42
0010143b 74 09  jz      00101444
0010143d b6 0c  mov     esi, 0c
00101442 eb 08  jmp     0010144c
00101444 b6 01  mov     esi, 01
00101446 00 0c  mov     esi, 0c
00101449 5d  pop     ebp
0010144a 90  ret
  
```

Decompiled Code View (Right Pane):

```

1  undefined8 check(char *param_1,ulong param_2)
2
3
4  {
5      undefined8 uVar1;
6      int local_c;
7
8      local_c = 0;
9      while( true ) {
10         if (param_2 <= (ulong)(long)local_c) {
11             if ((int)param_1[3] + (int)*param_1 == 0x72) {
12                 if ((int)param_1[0x12] + (int)param_1[1] == 0xd6) {
13                     if ((int)param_1[4] + (int)param_1[2] == 0xb2) {
14                         if ((byte)param_1[6] ^ param_1[5] == 0x4c) {
15                             if ((int)param_1[8] - (int)param_1[7] == 0x11) {
16                                 if ((int)param_1[10] - (int)param_1[9] == 0x3b) {
17                                     if (((int)param_1[0xb] + (int)param_1[0xc]) - (int)param_1[0xd] == 0x47) {
18                                         if (((int)param_1[0xe] + (int)param_1[0xf]) - (int)param_1[0x10] == 0x1f) {
19                                             if (((int)param_1[0x11] + (int)param_1[0x10]) - (int)param_1[0x12] == 0x58) {
20                                                 if ((byte)param_1[0x15] ^ param_1[0x13] ^ param_1[0x14] == 0x43) {
21                                                     uVar1 = 1;
22                                                 }
23                                             }
24                                             else {
25                                                 uVar1 = 0;
26                                             }
27                                         }
28                                         else {
29                                             uVar1 = 0;
30                                         }
31                                     }
32                                     else {
33                                         uVar1 = 0;
34                                     }
35                                 }
36                             }
37                         }
38                     }
39                 }
40             }
41         }
42     }
43 }
  
```

The status bar at the bottom shows the current instruction address as 0010143D, the instruction as 'check', and the register MOV EAX, 0x0.

now we can use `z3` to create a script which can generate a key that satisfies all of these conditions.

The following script does the same.

```
#!/usr/bin/env python3
from z3 import *

key = [BitVec(f'key_{i}', 8) for i in range(0x16)]

solver = Solver()

for i in range(0x16):
    solver.add(And(key[i] >= ord('0'), key[i] <= ord('z')))

solver.add(key[3] + key[0] == 0x72)
solver.add(key[0x12] + key[1] == 0xd6)
solver.add(key[4] + key[2] == 0xb2)
solver.add(key[6] ^ key[5] == 0x4c)
solver.add(key[8] - key[7] == 0x11)
solver.add(key[10] - key[9] == 0x3b)
solver.add(key[0xb] + key[0xc] - key[0xd] == 0x47)
solver.add(key[0xe] + key[0xf] - key[0x10] == 0x1f)
solver.add(key[0x11] + key[0x10] - key[0x12] == 0x58)
solver.add(key[0x15] ^ key[0x13] ^ key[0x14] == 0x43)

if solver.check() == sat:
    m = solver.model()
    flag = ''.join([chr(m.evaluate(key[i]).as_long()) for i in range(0x16)])
    print(flag)
```

Running the script we get the key as `8]x::x49J8s8K<8^wZy9J0`

now we can get the flag by sending the key to the endpoint.

```
> nc 35.184.207.253 1337
Enter key: 8]x::x49J8s8K<8^wZy9J0
Congrats here is your flag: ctf{z3_f0r_l!f3}
|
```

An Edutech product of
RedTeam Hacker Academy Pvt. Ltd.

PentestGorage

REDTEAM360
AMALGAMATING CYBERSECURITY EDUCATION

RedTeam Hacker Academy Pvt. Ltd.

An ISO 9001 - 2015 Certified Company

Contact

PUNE

Level 4, Prabhavatech Park,
Baner, Pune, Maharashtra 411045A

+91 859 060 4105

TRIVANDRUM

2nd Floor, Athens Plaza, SS Kovil Road,
Near Apollo Dimora, Opposite
Central Railway Station, Thampanoor,
Thiruvananthapuram, Kerala 695001

+91 956 219 5666

MANGALORE

2nd Floor, Alfa Towers,
Balmatta Rd, Hampankatta,
Mangalore, Karnataka 575001

+91 789 923 4666

KOCHI

1st Floor, Kariyappilli Tower,
Near Kinder Hospital, Metro Pillar
No.345, VMB Road,
Pathadippalam, Edappally,
Ernakulam, Kerala 682024

+91 956 290 4666

CALICUT

3rd Floor, Thayyil Arcade,
Opposite Baby Memorial Hospital,
V. Panoli Road, Azhakodi Temple Road,
Kozhikode, Kerala 673004

+91 956 250 3666

KOTTAKKAL

4th Floor, K.P.M Tower,
Above South Indian Bank
Tirur, Malappuram, Manjeri Rd,
Kottakkal, Kerala 676503

+91 984 629 1666



@redteamacademy